Embedded Data Structure
(Lecture Note 25)

Searching Algorithms (8)

2025. 6. 12

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

R-B Tree : Implement (1/5)

▪ C implementation – cont'd

- **Rotation (key → direction)**

```
node *rotate(int key, node *pivot, node *base)
{
    node *child, *gchild;
    if(key < pivot->key || pivot == head)
        child = pivot->left;
    else
        child = pivot->right;
    if(key < child->key){    // L rotation
        gchild = child->left;
        child->left = gchild->right;
        gchild->right = child;
    }
    else{                    // R rotation
        gchild = child->right;
        child->right = gchild->left;
        gchild->left = child;
    }
    if(key < pivot->key || pivot
    == base)
        pivot->left = gchild;
    else
        pivot->right = gchild;

    return gchild;
}
```

R-B Tree : Implement (2/5)

- **C implementation – cont'd**

  - **Insertion function**

```
node *rbti_insert(int key, node *base, int *num)
{
    node *t, *parent, *gparent, *ggparent;
    ggparent = gparent = parent = base;
    t = base->left;
    while(t != NULL){
        if(key == t->key) return NULL;
        if(t->left && t->right && t->left->red && t->right->red){
            t->red = 1;
            t->left->red = t->right->red = 0;
            if(parent->red){
                gparent->red = 1;
                if(key < gparent->key != key < parent->key)
                    parent = rotate(key, gparent, base);
                t = rotate(key, ggparent, base);
                t->red = 0;
            }
```

What does it mean ?

What does it mean ?

Rotation is required !

LR or RL

R-B Tree : Implement (3/5)

- **C implementation – cont'd**
  - **Insertion function – cont'd**

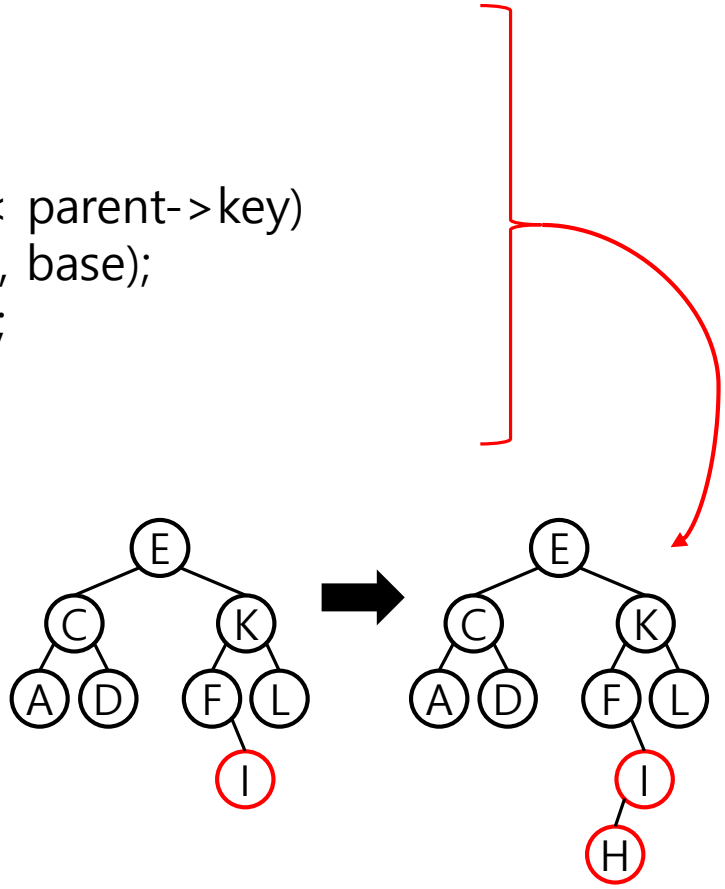
```
        base->left->red = 0;
    }
    ggparent = gparent;
    gparent = parent;
    parent = t;
    if(key < t->key) t = t->left;
    else t = t->right;
}
t = (node *)malloc(sizeof(node));
t->key = key;
t->left = NULL;
t->right = NULL;
if(key < parent->key || parent == base) parent->left = t;
else parent->right = t;
(*num)++;
t->red = 1;
```

C implementation – cont'd

- **Insertion function – cont'd**

```
    if(parent->red)
    {
        gparent->red = 1;
        if(key < gparent->key != key < parent->key)
            parent = rotate(key, gparent, base);
        t = rotate(key, ggparent, base);
        t->red = 0;
    }
    base->left->red = 0;

    return t;
}
```



['H' node insertion]

R-B Tree : Implement (5/5)

- **C implementation – cont'd**
  - **Insert K L C A E F D I H G and check the result !**

Make your code complete !

Summary

- **Sorting and Searching**
  - **Think about the story of this class !**